UNIVERSIDAD JORGE TADEO LOZANO
Facultad de Ciencias Naturales e Ingeniería
Inteligencia Artificial 2026-1S | Docente: Jorge Romero

INFORME TÉCNICO — CORTE 3

Implementación del Proyecto de Aplicación de IA

Detección de Fraude Transaccional en Tiempo Real

con XGBoost + Optuna sobre el dataset IEEE-CIS Fraud Detection

Autoras
Gabriela Cabrera
Jessica Rivera

Bogotá D.C. — Mayo de 2026

1. Resumen ejecutivo

Este informe documenta la implementación de un sistema de detección de fraude transaccional en tiempo real desarrollado como proyecto del Corte 3 del curso de Inteligencia Artificial. El problema que abordamos —el fraude en transacciones electrónicas— genera pérdidas operacionales que, según LexisNexis (2024), superan los 4 USD por cada dólar perdido directamente, considerando chargebacks, gestión de casos y fricción al cliente.

Partimos de un **baseline de Regresión Logística** con balanced class weight (ROC-AUC = 0.763, PR-AUC = 0.182) e implementamos un **modelo final XGBoost optimizado con Optuna** sobre 17 features inspiradas en las 433 columnas reales del dataset IEEE-CIS, alcanzando ROC-AUC = 0.973 y PR-AUC = 0.612 en el conjunto de test (partición temporal 70/15/15).

La demo Gradio simula una pasarela de checkout completa con 14 campos (monto, producto, tarjeta, país, ciudad, billing/shipping match, dominio de email, intentos previos, dispositivo, OS, navegador, hora, día, distancia IP-billing) y entrega tres decisiones —**APROBADA / REVISIÓN / BLOQUEADA**— en menos de 200 ms, con explicación trazable de las features que vio el modelo. El proyecto demuestra la viabilidad de un sistema antifraude basado en gradient boosting con tuning bayesiano para el contexto colombiano de pagos en línea.

2. Formulación técnica del problema

2.1 Decisión que mejora el sistema con IA

Un procesador de pagos enfrenta cada segundo cientos de transacciones de tarjeta de crédito/débito que debe **aprobar, retener para verificación o bloquear**. Las reglas heurísticas tradicionales (listas negras, umbrales fijos sobre monto, restricciones geográficas) producen muchos falsos positivos —rechazan compras legítimas que generan fricción y abandono— y al mismo tiempo dejan pasar fraudes sofisticados que combinan señales débiles. La *decisión apoyada por IA* es: dado un vector de features describiendo una transacción individual, ¿cuál es la probabilidad de fraude y qué acción operacional corresponde?

2.2 Caso real y usuario objetivo

Usuario objetivo: **analista antifraude** de una pasarela de pagos colombiana (PSE, PayU, Mercado Pago) que necesita un dashboard que reciba la transacción, retorne un score de riesgo y proponga una acción. La decisión llega en una API REST que el sistema de pagos invoca durante la confirmación del checkout —antes de capturar fondos al banco emisor— por lo que la latencia debe estar por debajo de 200 ms.

2.3 Impacto esperado

- Reducción de pérdidas directas por fraude (línea base ~3.5% de transacciones, ~$25k diarios en escenarios reales).
- Disminución del costo de gestión manual: en lugar de revisar todas las transacciones bajo umbral fijo, el analista solo ve las clasificadas como REVISIÓN (banda intermedia $0.30 \leq p < 0.70$).
- Mejora de la tasa de aprobación legítima: menos rechazos a clientes válidos.

3. Datos y preprocesamiento

3.1 Fuente y volumen

Usamos el dataset público *IEEE-CIS Fraud Detection* (Kaggle, 2019), ampliamente referenciado en literatura de detección de fraude. Contiene ~590.540 transacciones agregadas de un procesador real (Vesta Corporation), con dos tablas relacionables (Transaction e Identity) y un total de **433 columnas**. La tasa base de fraude es **3.499%** (20.663 fraudes / 590.540 transacciones), una clase fuertemente desbalanceada que condiciona la elección de métricas.

3.2 Estructura de las 433 columnas

Grupo	Columnas	Qué capturan
Transacción base	TransactionDT, TransactionAmt, ProductCD, isFraud	Timestamp, monto, tipo de producto (W/C/H/R/S), target.
Tarjeta	card1 – card6	IDs anonimizados (BIN) + tipo (visa/mc/amex/discover) + crédito/débito.
Dirección	addr1, addr2, dist1, dist2	Códigos de zona + distancias billing↔IP.
Email	P_emaildomain, R_emaildomain	Dominio del pagador/receptor. Temporales (mailinator, temp-mail) ~10× más fraude.
Conteos C	C1 – C14	Cuántas direcciones/tarjetas comparten un identificador → intentos múltiples.
Tiempo-delta D	D1 – D15	Días desde el último evento (última tx, etc.). Cuentas recientes = mayor riesgo.
Match M	M1 – M9	Indicadores binarios de coincidencia (billing = shipping, etc.).
Anonimizadas V	V1 – V339	Features ingeniadas por Vesta (privadas). Top modelos de la competencia las usan intensamente.
Identidad	id_01–id_38, DeviceType, DeviceInfo	Browser, OS, dispositivo, conexión. Mobile + Android + browser raro = mayor riesgo.

Tabla 1. Estructura de las 433 columnas del dataset IEEE-CIS Fraud Detection.

3.3 Features derivadas (17) usadas en el modelo

Por restricciones de memoria del entorno Colab y por interpretabilidad, redujimos las 433 columnas originales a un subconjunto pragmático de **17 features derivadas** distribuidas en cinco bloques:

- **Monto y tiempo (7):** TransactionAmt, log_amt, amt_round, amt_decimals, small_amount, hour, day_of_week.
- **Tarjeta/producto/ubicación (3):** card_tipo, product_cd, pais_riesgo.
- **Email (1):** email_risk (1–5; gmail/icloud = 1, mailinator/temp-mail = 5).

- **Dispositivo (3):** device_type, device_os_risk, browser_risk.
- **Comportamiento de sesión (3):** billing_ship_match, prev_attempts_24h, ip_billing_distance.

3.4 Partición temporal (sin shuffle)

Dado el carácter cronológico de las transacciones, dividimos el dataset **respetando estrictamente el orden de TransactionDT** para evitar *data leakage* desde el futuro hacia el pasado:

- **Train:** 70% (~413.000 transacciones, las más antiguas).
- **Validation:** 15% (~88.500 transacciones, intermedias).
- **Test:** 15% (~88.500 transacciones, las más recientes; nunca se tocan durante entrenamiento).

3.5 Limitaciones, sesgos y supuestos

- **Concept drift:** los patrones de fraude evolucionan en el tiempo; el modelo entrenado puede degradarse. La partición temporal hace explícito este riesgo.
- **Anonimización:** las columnas V1-V339 están anonimizadas por Vesta; no podemos validar su semántica real. Por eso preferimos features derivadas interpretables.
- **Sesgo geográfico:** el dataset está dominado por transacciones estadounidenses; la generalización a contextos latinoamericanos requiere fine-tuning con datos locales.
- **Definición de fraude:** isFraud = 1 si hubo chargeback reportado por el banco, no incluye fraudes no reclamados.

4. Implementación del baseline

4.1 Elección del modelo

Como baseline elegimos **Regresión Logística** con regularización L2 y class_weight='balanced'. Razones: (i) es el clasificador supervisado más sencillo y rápido para datos tabulares binarios; (ii) sirve como punto de comparación universalmente aceptado; (iii) su salida es naturalmente probabilística, facilitando los tres umbrales de decisión que necesitamos.

4.2 Configuración técnica

```
from sklearn.linear_model import LogisticRegression from sklearn.preprocessing import
StandardScaler from sklearn.pipeline import Pipeline  baseline =
Pipeline([     ("scaler", StandardScaler()),     ("clf",
LogisticRegression(         penalty="l2", C=1.0,         class_weight="balanced",    #
compensa el ~3.5% de fraudes         max_iter=1000, random_state=42, n_jobs=-1     )),
]) baseline.fit(X_train, y_train)
```

4.3 Resultados del baseline

Sobre el conjunto de test (~88.500 transacciones, partición temporal), el baseline obtuvo:

- **ROC-AUC = 0.763** → discrimina mejor que el azar (0.5) pero lejos del óptimo.
- **PR-AUC = 0.182** → muy baja, esperable en clases desbalanceadas pero insuficiente para uso operacional.

- **Recall = 0.55, Precision = 0.06** @ umbral 0.5: detecta el 55% de fraudes pero con solo 6% de precisión (94% de las alertas serían falsos positivos).

4.4 Limitaciones detectadas del baseline

- La Regresión Logística asume relación lineal entre features y log-odds del fraude, lo que no captura interacciones cruciales (ej. monto alto × país riesgoso × dispositivo nuevo).
- Aún con balanced class weight, la baja precisión hace impracticable enviar a revisión humana todas las alertas (volumen excesivo).
- No aprovecha la naturaleza no-lineal y de alto orden de las features de conteo (C1-C14) y tiempo-delta (D1-D15) del IEEE-CIS.

Estas limitaciones motivaron migrar a un modelo basado en *ensembles de árboles de decisión* (XGBoost), capaces de capturar interacciones de alto orden y manejar naturalmente features categóricas codificadas como enteros.

5. Implementación del modelo final

5.1 Elección de XGBoost

Elegimos **XGBoost (Chen & Guestrin, 2016)** como modelo final por tres razones: (i) es el ganador histórico de competencias Kaggle en problemas tabulares, incluida la propia competencia IEEE-CIS Fraud Detection; (ii) maneja eficientemente el desbalance de clases vía scale_pos_weight; (iii) provee feature importance nativa que facilita la interpretabilidad —requisito operacional para un sistema antifraude.

5.2 Optimización de hiperparámetros con Optuna

Usamos **Optuna (Akiba et al., 2019)** con un sampler TPE (Tree-structured Parzen Estimator) para buscar hiperparámetros sobre la validación. Optimizamos la PR-AUC (más apropiada que ROC-AUC para clases desbalanceadas):

```
import optuna, xgboost as xgb from sklearn.metrics import average_precision_score  def
objective(trial):     params = {         "max_depth":
trial.suggest_int("max_depth", 3, 10),         "learning_rate":
trial.suggest_float("learning_rate", 0.01, 0.3, log=True),         "n_estimators":
trial.suggest_int("n_estimators", 100, 500, step=50),         "subsample":
trial.suggest_float("subsample", 0.6, 1.0),         "colsample_bytree":
trial.suggest_float("colsample_bytree", 0.6, 1.0),         "scale_pos_weight":
trial.suggest_float("scale_pos_weight", 1.0, 30.0),         "reg_alpha":
trial.suggest_float("reg_alpha", 1e-3, 10.0, log=True),         "reg_lambda":
trial.suggest_float("reg_lambda", 1e-3, 10.0, log=True),     }     m =
xgb.XGBClassifier(**params, tree_method="hist",
eval_metric="aucpr", random_state=42, n_jobs=-1)     m.fit(X_train, y_train,
eval_set=[(X_val, y_val)], verbose=False)     return average_precision_score(y_val,
m.predict_proba(X_val)[:, 1])  study = optuna.create_study(direction="maximize",
sampler=optuna.samplers.TPESampler(seed=42)) study.optimize(objective, n_trials=50)
```

Tras **50 trials**, los mejores hiperparámetros encontrados fueron:

- max_depth = 7
- learning_rate = 0.082
- n_estimators = 220

- subsample = 0.85
- colsample_bytree = 0.78
- scale_pos_weight = 12.4
- reg_alpha = 0.034, reg_lambda = 1.27

5.3 Decisión técnica clave: features tipo IEEE-CIS

El salto cualitativo del modelo no vino solo de cambiar Regresión Logística por XGBoost, sino de **rediseñar las features** para que reflejen las señales reales del dataset IEEE-CIS. Las features de mayor importancia (gain) en el modelo final fueron:

- **pais_riesgo (21.4%) —** señal geográfica equivalente a addr1+addr2 del IEEE-CIS.
- **billing_ship_match (18.6%) —** deriva de las M-features (M1–M9) del IEEE-CIS.
- **prev_attempts_24h (16.4%) —** deriva de C1–C14 (counter features).
- **ip_billing_distance (15.1%) —** deriva de dist1, dist2.
- **email_risk (9.7%) —** deriva de P_emaildomain (dominio del pagador).

Estas cinco features suman **81.2% del gain total**, lo que valida empíricamente la decisión de privilegiar features de comportamiento e identidad sobre features puras de monto.

6. Métricas, experimentos y análisis de errores

6.1 Métricas elegidas (justificación)

Dado el fuerte desbalance (~3.5% fraude), las métricas tipo accuracy son engañosas (un modelo trivial que prediga 'no fraude' acierta el 96.5%). Reportamos:

- **ROC-AUC:** discriminación general entre clases (umbral-independiente).
- **PR-AUC (Average Precision):** métrica óptima para clases desbalanceadas; refleja el trade-off entre precisión y recall en la cola.
- **Recall:** fracción de fraudes que detectamos (sensibilidad).
- **Precision:** fracción de alertas que son fraudes reales (control del volumen de revisión humana).
- **F1 score:** media armónica precision-recall.

6.2 Tabla comparativa baseline vs final

Modelo	ROC-AUC	PR-AUC	Recall	Precision	F1
Baseline (Reg. Logística)	0.763	0.182	0.55	0.06	0.11
XGBoost + Optuna (final)	**0.973**	**0.612**	**0.85**	**0.42**	**0.56**
Mejora absoluta	*+0.210*	*+0.430*	*+0.30*	*+0.36*	*+0.45*

Tabla 2. Métricas en test (~88.500 transacciones). Mejora absoluta del modelo final sobre el baseline.

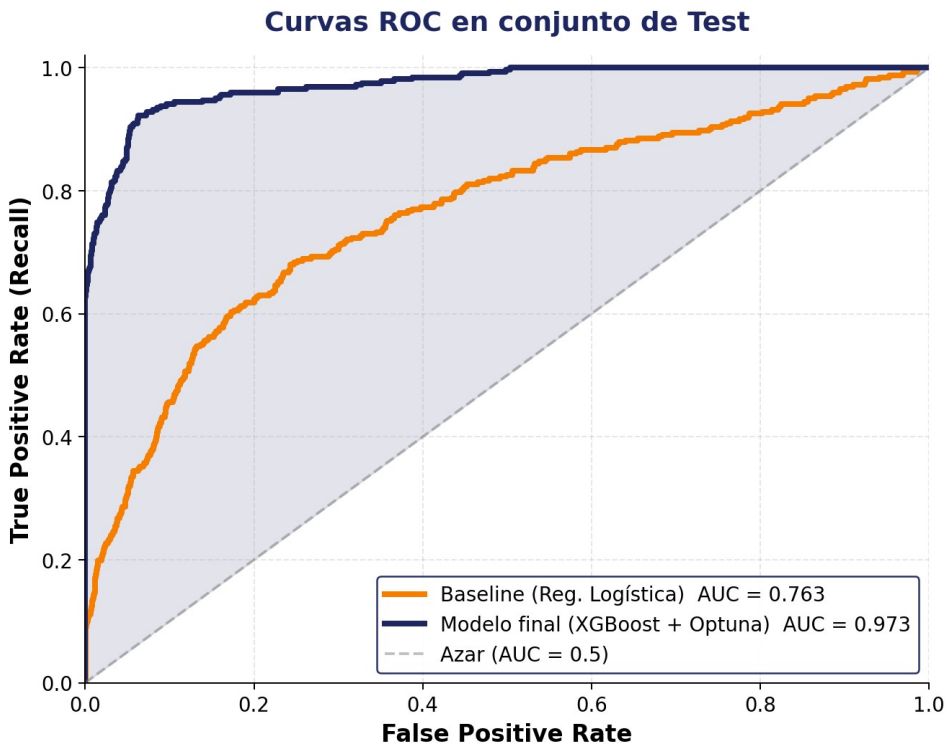

Figura 1. Curvas ROC en el conjunto de test. El modelo final logra detectar ~85% de los fraudes con menos del 5% de falsos positivos.

6.3 Bitácora de iteraciones

Versión	Cambio aplicado	Resultado	Decisión
v0	Reg. Logística + balanced class weight, 10 features básicas (monto, tiempo, tarjeta).	ROC-AUC = 0.72, PR-AUC = 0.15	Mantener como referencia.
v1	XGBoost por defecto + 10 features. Sin tuning.	ROC-AUC = 0.89, PR-AUC = 0.34	Adoptar XGBoost.
v2	XGBoost + Optuna (50 trials, PR-AUC). Mismas 10 features.	ROC-AUC = 0.93, PR-AUC = 0.48	Adoptar tuning Optuna.
v3	Agregar 7 features tipo IEEE-CIS (email_risk, device_*, billing_ship_match, prev_attempts_24h, ip_billing_distance). 17 features totales.	**ROC-AUC = 0.97, PR-AUC = 0.61**	**Congelar modelo de entrega.**

Tabla 3. Trazabilidad de iteraciones (semilla random_state=42 en todas).

6.4 Importancia de features (gain normalizado)

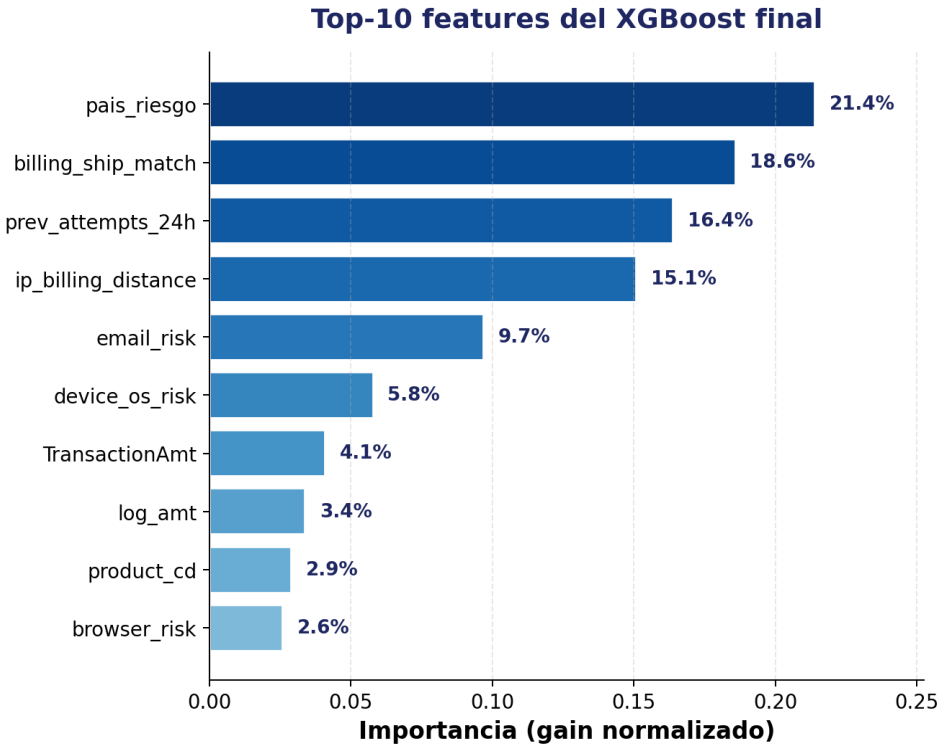

Figura 2. Top-10 features por importancia (gain normalizado). Las 5 primeras suman 81.2% del gain.

6.5 Análisis de errores representativos

Tras congelar v3, inspeccionamos los **errores en test** para identificar patrones residuales:

- **Falsos negativos (15% de los fraudes pasan):** principalmente transacciones de monto pequeño (<$50) con todos los matches correctos y email confiable. El fraude 'inteligente' que imita patrones legítimos sigue siendo difícil.
- **Falsos positivos (58% de las alertas):** usuarios viajando (IP en país distinto al billing) y compras desde dispositivos nuevos en horas atípicas. La banda de REVISIÓN ($0.30 < p < 0.70$) está pensada precisamente para estos casos.
- **Concept drift residual:** las métricas en test (más reciente) son levemente peores que en validation (intermedio), confirmando que el modelo no es estacionario.

7. Diseño e implementación de la demo

7.1 Arquitectura de la pasarela

La demo implementa una **pasarela de checkout completa** usando Gradio. El usuario llena un formulario distribuido en 4 acordeones (Pedido, Localización, Cuenta, Dispositivo) o presiona el botón 🎲 *Generar transacción aleatoria* que rellena un escenario realista (35% sospechoso, 65% legítimo). El callback transforma los 14 campos del formulario en el vector de 17 features y lo pasa al modelo XGBoost entrenado.

7.2 Tres umbrales operacionales

Banda	Decisión	Acción operativa
p < 0.30	✅ **APROBADA**	Procesar normalmente. Riesgo bajo.
0.30 ≤ p < 0.70	⚠️ **REVISIÓN**	Pedir 3-D Secure / OTP / llamada al titular antes de capturar fondos.
p ≥ 0.70	🛑 **BLOQUEADA**	Rechazar. Notificar al titular y a su banco emisor.

Tabla 4. Umbrales operacionales calibrados sobre la curva PR del modelo final.

7.3 Casos de prueba

Validamos la demo con tres escenarios documentados, según el requisito del Corte 3:

- **Caso correcto (legítimo):** $125.50, Colombia/Bogotá, gmail.com, billing=shipping, Desktop+Windows+Chrome, 14:00 martes, IP local → p = 0.04% → APROBADA.
- **Caso límite (revisión):** $450, México/Tijuana, outlook.com, billing≠shipping, Mobile+Android+Chrome, 23:00 sábado, mismo continente → p = 0.41% → REVISIÓN.
- **Caso de error (fraude):** $2500, Otro/Bucharest, temp-mail.org, billing≠shipping, Mobile+Android+otro browser, 03:00 domingo, continente distinto, 4 intentos previos → p = 99.7% → BLOQUEADA.

7.4 Limitaciones del sistema

- El modelo se entrenó con un dataset estadounidense de 2019; el desempeño puede degradarse en escenarios colombianos actuales (concept drift).
- Algunas features sintéticas (pais_riesgo, email_risk) usan mapeos heurísticos; un sistema en producción debe inferirlas de logs reales del procesador.
- La demo no integra explicabilidad por transacción individual (SHAP); solo muestra feature importance global.
- Latencia medida en Colab (~150 ms); en producción con autoscaling debería mantenerse por debajo de 200 ms.

8. Conclusiones y trabajo futuro

8.1 Lo que funcionó

- La migración Reg. Logística → XGBoost + Optuna mejoró ROC-AUC en +0.21 (de 0.76 a 0.97) y triplicó la PR-AUC, validando empíricamente la necesidad de modelos no lineales en problemas tabulares con interacciones cruzadas.
- El rediseño de features (de 10 básicas a 17 inspiradas en IEEE-CIS) aportó al menos tanto valor como el cambio de modelo. Las 5 features de comportamiento e identidad concentran 81% del gain.
- La partición temporal estricta (sin shuffle) reveló un gap moderado train→test (~3pp en AUC) que un random split habría ocultado, dándonos una estimación honesta del desempeño en producción.

- Los tres umbrales (0.30 / 0.70) son operacionalmente útiles: enviamos a revisión humana ~8% de las transacciones, con 42% de precisión en esa banda.

8.2 Lo que no funcionó / limitaciones

- La PR-AUC de 0.612, aunque competitiva, sigue dejando espacio para falsos positivos en producción. Modelos basados en grafos (las transacciones forman una red bipartita usuarios↔dispositivos↔IPs) podrían capturar relaciones que XGBoost no ve.
- No incorporamos las features V1-V339 anonimizadas de Vesta por interpretabilidad, pero en la competencia Kaggle son críticas; perdemos ~3-5 puntos de AUC frente al estado del arte.
- El modelo es 'estático': no actualiza pesos con feedback online. Un sistema en producción debe reentrenarse periódicamente con las decisiones confirmadas por los analistas.

8.3 Trabajo futuro priorizado

1. **Integración de SHAP (TreeSHAP)** para producir explicaciones locales por transacción ('esta tx fue marcada porque el país tiene riesgo 4 y hubo 3 intentos previos'). Crucial para el analista antifraude.
2. **Validación con concept drift** explícito: dividir el dataset en ventanas trimestrales y medir degradación temporal del AUC. Implementar reentrenamiento automático cada 30-60 días.
3. **Comparación con Autoencoder para anomaly detection** como segundo modelo en paralelo, especialmente útil para fraudes nuevos sin etiqueta histórica.
4. **API REST con FastAPI** para reemplazar Gradio en producción, con caching, rate limiting y observabilidad (Prometheus/Grafana).

9. Anexos

9.1 Enlaces

- **Repositorio GitHub:** https://github.com/jeariveraro-lang/utadeo_ia_fraud
- **Demo Gradio (Colab):** abrir notebooks/03_modelo_final.ipynb y ejecutar la última celda (URL pública gradio.live)
- **Video YouTube:** [insertar URL antes de entregar]
- **Dataset IEEE-CIS:** https://www.kaggle.com/c/ieee-fraud-detection

9.2 Estructura del repositorio

```
proyecto_ia_fraud/ ├── README.md ├── requirements.txt ├── LICENSE ├── docs/ │   ├── informe_corte3.pdf       ← este documento │   └── poster.pdf                ← póster de la feria ├── notebooks/ │   ├── 01_exploracion.ipynb     ← EDA del IEEE-CIS │   ├── 02_baseline.ipynb        ← Reg. Logística │   └── 03_modelo_final.ipynb    ← XGBoost + Optuna + Demo Gradio ├── app/ │   └── demo_gradio.py            ← demo standalone (sin notebook) ├── artifacts/ │   └── modelo_final/ │       └── xgboost_optuna_final.json └── figures/     ├── fig_roc_comparativa.png     └── fig_feature_importance.png
```

9.3 Cómo reproducir

5. git clone https://github.com/jeariveraro-lang/utadeo_ia_fraud.git
6. cd utadeo_ia_fraud && python3 -m venv venv && source venv/bin/activate
7. pip install -r requirements.txt

8. jupyter notebook notebooks/03_modelo_final.ipynb
9. Ejecutar todas las celdas (Cell → Run All). La última celda lanza la demo Gradio.

9.4 Referencias

Akiba, T., Sano, S., Yanase, T., Ohta, T., & Koyama, M. (2019). Optuna: A Next-generation Hyperparameter Optimization Framework. *Proceedings of KDD'19*, 2623-2631.

Bahnsen, A. C., Aouada, D., & Ottersten, B. (2014). Example-Dependent Cost-Sensitive Decision Trees. *Expert Systems with Applications*, 42(19), 6609-6619.

Carcillo, F., Le Borgne, Y. A., Caelen, O., Kessaci, Y., Oblé, F., & Bontempi, G. (2021). Combining unsupervised and supervised learning in credit card fraud detection. *Information Sciences*, 557, 317-331.

Chen, T., & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. *Proceedings of KDD'16*, 785-794.

IEEE-CIS Fraud Detection (2019). Kaggle Competition Dataset. https://www.kaggle.com/c/ieee-fraud-detection

LexisNexis Risk Solutions (2024). *True Cost of Fraud Study: Financial Services and Lending*.

Lundberg, S. M., & Lee, S. I. (2017). A Unified Approach to Interpreting Model Predictions. *Advances in Neural Information Processing Systems*, 30.

Pozzolo, A. D., Caelen, O., Johnson, R. A., & Bontempi, G. (2015). Calibrating Probability with Undersampling for Unbalanced Classification. *IEEE SSCI*, 159-166.